SKALA

AI 서비스 개발 Mini-project

상용 서비스 아키텍처 기반 AI 서비스 개발 · 3일 과정

임성열 교수 Ph.D. | 2026.10

SK 주식회사 AX

과정 안내

3일 과정 전체 구성

ROADMAP

기간	세션	주요 내용	시간	방식
Day 1	AI 서비스 기획	AI 서비스의 성패는 어디서 갈리는가 · 5가지 분석 틀 · 대비 사례 3건	1	이론
	상용 서비스 아키텍처	Vue.js – Java 인증서버(JWT) – FastAPI AI 서비스 – LLM, MSA 구조	1	이론
	데모·연결	Vue.js 반응형 UX · Java 인증 · 음성 통번역(Physical AI) · External RAG 연결	2	데모
	주제 탐색·설계·착수	서비스 유형 택1 · 기획서·아키텍처·API 명세 · 체크포인트①②	4	프로젝트
Day 2	조별 구현	데이터 파이프라인 → RAG 서비스 → 인증 연동 API → Vue.js 연동 · 체크포인트③④	8	프로젝트
Day 3	품질·배포·발표	평가셋 기반 개선 → Docker Compose 배포 → 운영 점검 → 퀴즈·발표 · 체크포인트⑤	8	프로젝트·평가

! **핵심 원칙** Streamlit·Colab 같은 PoC 도구가 아니라, 실제 상용 서비스와 같은 계층 분리 구조로 끝까지 만들어 본다.

과정 안내

과정 목표와 평가 방식

GOAL · EVALUATION

PoC를 넘어, 상용 서비스 구조로 AI 서비스를 끝까지 완성한다

01

기획·설계

상용 아키텍처(Vue.js – REST API – LLM) 기반으로 문제를 정의하고 서비스를 설계

02

구현

제공된 골격(인증·프론트) 위에 External RAG 서비스를 직접 구현해 연결

03

품질·배포

평가셋 기반 품질 개선, 컨테이너 배포·운영 점검, 발표까지 전 과정 완주

구분	평가 항목 (배점)	비고
개인 30	퀴즈 15문항: 아키텍처·인증 · RAG·모델 서빙 · 평가·배포·운영 (각 10)	Day3 공통 문항
조별 70	기획·설계 10 · 구현 20 · 평가/개선 10 · 서비스화·배포 15 · 발표 10 · 과정 5	+ 확장 가산 5
개인 계수	조별 점수 × 기여 계수 0.3~1.0 (역할 이행 · 체크포인트 설명 · 개별 질의응답)	100점 상한

Day 1 · AI 서비스 기획과 상용 아키텍처

- ① AI 서비스 기획 – 성패는 어디서 갈리는가
- ② 상용 서비스 아키텍처
- ③ 예제 실습 – Internal / External RAG
- ④ 주제 탐색·설계 ⑤ 프로젝트 착수

SK 주식회사 AX

AI 서비스 기획

DAY 1 :: STAGE 1/5

①서비스 기획	②상용 아키텍처	③데모·연결	④주제·설계	⑤착수

같은 모델, 다른 결과

--question · 5분

같은 모델을 쓰는데,
왜 어떤 서비스는 성공하고 어떤 서비스는 사라지는가?

- 최근 한 달 동안 실제로 돈을 내고 쓴 AI 서비스는 무엇인가? 왜 그 서비스였는가?
- 써 보다가 그만둔 AI 서비스는 무엇인가? 그만둔 이유가 모델 성능 때문이었는가?
- 같은 LLM API를 쓰는 사내 챗봇 두 개 중, 하나만 계속 쓰이는 이유는 무엇일까?
- **개발자 질문:** 그 서비스를 API로 상상해 보자 - 요청(입력)과 응답(출력)은 무엇이고, 틀리면 어떻게 되는가?

AI 서비스 기획

DAY 1 :: STAGE 1/5

①서비스 기획	②상용 아키텍처	③데모·연결	④주제·설계	⑤착수

오늘의 핵심 메시지와 학습 목표

핵심 질문

핵심 메시지

모델은 누구나 빌려 쓸 수 있는 시대의 경쟁력

- 모델 성능은 빠르게 평준화되고 있다
- 차이는 어떤 문제를 골랐는가에서 시작된다
- 그리고 오류 · 비용 · 책임을 어떻게 운영하는가에서 벌어진다
- **개발자 버전:** LLM 호출 코드보다 스키마 · 검증 · 로그 · 평가 코드가 더 길어진다

학습 목표

이 세션이 끝나면 할 수 있어야 하는 것

- AI 서비스 사례를 5가지 틀로 분해해 설명할 수 있다
- 성공 · 실패 사례의 차이를 틀의 언어로 말할 수 있다
- 각 틀을 API 스키마 · 평가 지표 · 로그 같은 구현 결정으로 번역할 수 있다
- 조별 기획서(체크포인트①)의 뼈대로 쓸 수 있다

AI 서비스 기획

DAY 1 :: STAGE 1/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | ④주제·설계 | ⑤착수

사례를 보는 5가지 틀

--theory · 40분

01

문제와 사용자

누구의 어떤 불편을 푸는가
→ 핵심 질문 · 기존 방식

02

AI의 역할

대체인가, 보조인가
→ needs_review · handoff

03

구현 선택

차별점은 어디에 있는가
→ Internal · External

04

품질과 위험

오류를 어떻게 관리하는가
→ 평가셋 · 출처

05

수익 구조

비용 대비 수익이 맞는가
→ 토큰 · 원가 로그

! 사례는 몇 달이면 낡지만 틀은 남는다. 틀마다 "그래서 코드에서는 무엇을 하는가"(→)까지 내려간다.

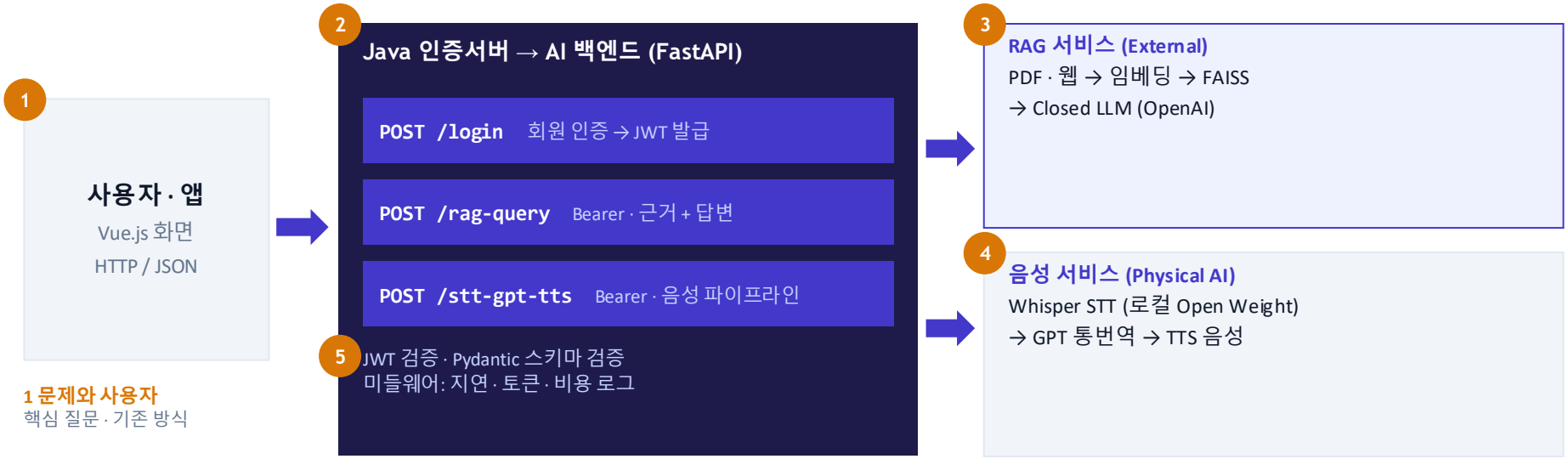
AI 서비스 기획

DAY 1 :: STAGE 1/5

①서비스 기획	②상용 아키텍처	③데모·연결	④주제·설계	⑤착수

우리 프로젝트 아키텍처에 5가지 틀 겹쳐 보기

REFERENCE



! **예시 · 음성 질의 RAG** 로그인 → 마이크로 "최신 AI 규제 동향은?" → STT → RAG(근거 · 출처) → TTS 음성 답변

AI 서비스 기획

DAY 1 :: STAGE 1/5

틀 1. 문제와 사용자 - 누구의 어떤 불편을 푸는가?

핵심 질문

- 사용자는 누구이며, 지금은 그 일을 어떻게 처리하고 있는가
- 그 불편은 얼마나 자주, 얼마나 크게 발생하는가
- AI 없이도 풀 수 있는 문제는 아닌가 - 검색 · FAQ · 담당자로 충분하지 않은가
- "AI를 넣을 곳"을 찾은 것인가, "풀어야 할 문제"를 찾은 것인가
- **개발자 질문:** 질문마다 기대답변과 근거 문서를 적을 수 있는가

DEV 개발로 번역하면

```
# 1) 핵심 질문 10개 + 기대답변 + 근거
Q01 = {"q": "연차 이월 기준은?",
       "expected": "최대 5일",
       "source": "인사규정 12조"}

# 2) AI 없이 풀리는가? 베이스라인 먼저
baseline = keyword_search(q)   # 기존 검색
# RAG 정답률 > 베이스라인 일 때만
# AI를 넣을 이유가 생긴다
```

! 좋은 문제의 조건: 빈도가 높고, 지금의 해결 방식이 비싸거나 느리며, 결과의 좋고 나쁨을 사용자가 바로 판단할 수 있다.

AI 서비스 기획

DAY 1 :: STAGE 1/5

틀 2. AI의 역할 - 대체인가, 보조인가?

핵심 개념

구분	보조 (Copilot)	대체 (Autopilot)
AI가 하는 일	초안 · 제안을 만들고 사람이 확정	사람 개입 없이 작업을 끝까지 수행
오류 발생 시	사람이 걸러 낼 기회가 있음	오류가 그대로 고객 · 외부로 전달
책임 소재	최종 결정한 사용자	서비스 제공 기업으로 이동
적합한 업무	틀려도 고칠 수 있는 업무	틀렸을 때 비용이 작거나 검증이 자동화된 업무
API 설계	needs_review: true → 사람이 확정 후 반영	자동 실행 + 임계값 · 롤백 · 감사 로그 필수
사례 메모	GitHub Copilot - 회색 제안, 개발자가 Tab으로 확정	Zillow Offers - 예측 가격이 곧 매입 제안가

! **질문: 이 서비스에서 AI가 틀렸을 때, 그 결과는 누가 책임지는가?**
→ 응답 스키마에 sources · needs_review · handoff 를 넣어 "사람이 확정하는 지점"을 코드로 남긴다

AI 서비스 기획

DAY 1 :: STAGE 1/5

틀 3. 구현 선택 - 차별점은 어디에 있는가?

핵심 개념

모델 선택

Open Weight냐 Closed냐, 직접 서빙이냐 API냐

- **Open Weight LLM:** 통제 · 보안 · GPU와 품질 부담
- **Closed LLM API:** 빠른 출시 · 토큰 과금 · 의존도
- **임베딩:** bge-m3(로컬) vs OpenAI(API)
- **기준:** 데이터 보안 등급 vs 운영 비용 · 품질

차별점의 위치

모델이 같다면 무엇이 다른가

- **데이터:** 고유 데이터 → 사내 DB · 규정 · 매뉴얼
- **워크플로:** 업무 흐름 안 → 기존 시스템이 부르는 API
- **UX:** 확인 · 수정 쉬움 → 응답에 근거 · 출처
- **프로젝트 기준:** RAG에 넣을 "우리만의 데이터"를 먼저 정한다

! **확인 질문: 경쟁사가 같은 모델로 내일 똑같은 기능을 만들면, 이 서비스에는 무엇이 남는가?**
→ 남는 것은 모델이 아니라 데이터 · 문서 · 평가셋이다. 프로젝트도 여기서 차별점을 만든다

AI 서비스 기획

DAY 1 :: STAGE 1/5

틀 4. 품질과 위험 - 오류를 어떻게 관리하는가?

핵심 개념

위험 유형	무엇이 문제인가	구현에서의 관리 방식
환각 · 오답	그럴듯하지만 틀린 답을 사실처럼 제시	검색된 문서만 근거로 답 · 근거 없으면 "찾을 수 없음" · 출처 반환
개인정보	입력 · DB에 포함된 민감 정보	입력 마스킹 · 로그에 원문 저장 금지 · 외부 API 전송 범위 확인
저작권	수집 문서와 생성물의 권리 문제	RAG 문서의 출처 · 라이선스 기록, 생성물임을 표시
품질 측정	"잘 된다"를 무엇으로 증명하는가	평가셋 20쌍 정답률 · 근거성 · 환각률, 개선 전후 비교

! 데모에서 잘 되는 것과 매일 운영에서 잘 되는 것은 다르다 → 평가셋을 먼저 만들고, 코드를 바꿀 때마다 다시 돌린다.
사례: Air Canada 챗봇(2024) - 없는 환불 규정을 안내, 회사가 배상 판정을 받음

AI 서비스 기획

DAY 1 :: STAGE 1/5

틀 5. 수익 구조 - 호출 비용 대비 수익이 맞는가?

핵심 개념

과금 방식	구조	장점	주의점
구독	월 정액	예측 가능한 매출	많이 쓰는 사용자일수록 원가 증가
사용량 기반	호출 · 토큰만큼 과금	원가와 매출이 연동	고객이 비용을 예측하기 어려움
성과 기반	해결 건수 · 절감액 기준	가치와 가격이 일치	성과 측정과 합의가 어려움

- 사용자 1명이 늘 때 비용도 함께 느는 구조 - 기존 소프트웨어와 가장 다른 점
- **요청당 원가** = 입력 토큰 × 단가 + 출력 토큰 × 단가 + 임베딩 · GPU 서버 → 미들웨어로 요청마다 기록
- **사례:** GitHub Copilot - 월 10달러 구독인데 사용자당 월평균 20달러 이상 손실이라는 보도(WSJ, 2023)

AI 서비스 기획

DAY 1 :: STAGE 1/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | ④주제·설계 | ⑤착수

같은 틀로 보는 대비 사례 3건

--case · 15분

성공 사례

효과가 숫자로 드러나는 분야

- 후보: 코딩 보조, 고객 상담 자동화
- 선정 기준: 도입 전후 지표가 공개된 사례
- **선정: GitHub Copilot (미국) - LLM 보조**

실패 사례

기대만 컸거나 책임 문제가 생긴 경우

- 후보: AI의 자동 의사결정, 챗봇의 잘못된 안내
- 선정 기준: 실패 원인이 문서로 확인되는 사례
- **선정: Zillow Offers (미국) - 자동 결정**

논쟁 중인 사례

편의와 통제 사이의 긴장

- 후보: 사람을 대체하는 AI 상담 · 에이전트
- 선정 기준: 찬반 근거가 모두 분명한 사례
- **선정: Klarna AI 상담 (스웨덴) - RAG형 상담**

! 세 사례는 우리 설계 결정과 1:1로 대응한다: Copilot = 보조형 응답 · Zillow = needs_review · Klarna = handoff(사람에게 넘기기).

AI 서비스 기획

DAY 1 :: STAGE 1/5

사례 1. 성공 사례 - GitHub Copilot

CASE 1/3

CASE PROFILE

서비스명: GitHub Copilot

제공 기업 · 국가: GitHub(Microsoft) · 미국

출시 시기: 2021.6 프리뷰, 2022.6 정식

핵심 지표: 통제 실험에서 과제 완료 시간 55.8% 단축

출처: Peng et al.(2023), arXiv:2302.06590

→ 프로젝트 교훈: AI는 제안, 확정은 사람. 결과를 즉시 검증할 수 있는 곳에 넣는다

분석 틀	관찰 내용
문제와 사용자	개발자의 반복 코드 작성. 하루 수백 번 발생하고, 실행 · 테스트로 결과를 바로 판단 가능
AI의 역할	보조. 회색 제안을 개발자가 Tab으로 수락 · 수정하며, 최종 책임은 개발자
구현 선택	외부 LLM 기반. 차별점은 IDE 안 워크플로와 열린 파일 · 저장소 맥락 활용
품질과 위험	수락률로 품질 측정 · 공개 코드와 일치하는 제안 차단 옵션 제공
수익 구조	좌석당 월 구독. 사용량 많은 사용자의 원가 부담 → 고급 모델 사용량 제한 도입

토론 질문 이 서비스의 성공은 5가지 중 어느 틀에서 결정되었는가?

AI 서비스 기획

DAY 1 :: STAGE 1/5

사례 2. 실패 사례 - Zillow Offers

CASE 2/3

CASE PROFILE

서비스명: Zillow Offers (iBuying)

제공 기업 · 국가: Zillow Group · 미국

출시 시기: 2018 시작, 2021.11 종료 발표

핵심 지표: 2021년 3분기 재고 평가손 약 3억 400만 달러, 인력 약 25% 감축

출처: Zillow 2021년 3분기 실적 발표 (2021.11.2)

→ 프로젝트 교훈: AI 출력을 바로 행동에 연결하면 임계값 · 사람 승인을 둔다

분석 틀	관찰 내용
문제와 사용자	빨리 집을 팔고 싶은 매도인. 불편은 크지만 회사가 주택 재고를 직접 떠안는 구조
AI의 역할	사실상 대체. 가격 예측이 곧 매입 제안가가 되어 오차가 그대로 손실로
구현 선택	가격 예측 모델(Zestimate 계열)에 의존. 차별점이 "예측 정확도" 하나에 집중
품질과 위험	2021년 시장 급변(데이터 드리프트)에서 비싸게 매입, 불확실성 관리 실패
수익 구조	건당 마진이 얇아 몇 % 오차가 손익을 뒤집음. 규모를 키울수록 위험도 커짐

토론 질문 5가지 중 어디에서 먼저 무너졌는가?

AI 서비스 기획

DAY 1 :: STAGE 1/5

사례 3. 논쟁 중인 사례 - Klarna AI 상담

CASE 3/3

CASE PROFILE

서비스명: Klarna AI 상담 어시스턴트

제공 기업 · 국가: Klarna · 스웨덴

출시 시기: 2024년 초 글로벌 출시

핵심 지표: 첫 달 상담 230만 건(전체 채팅의 2/3), 해결 시간 11분 → 2분, 상담원 700명분

출처: Klarna 발표(2024.2), CEO 인터뷰 보도(2025.5)

→ 프로젝트 교훈: AI 응답과 "사람에게 넘기기" 경로를 함께 설계한다

분석 틀	관찰 내용
문제와 사용자	결제 · 환불 · 분쟁 문의 대량 처리. 반복적 · 정형 문의가 많아 자동화에 적합
AI의 역할	대체 지향 → 품질 논란 후 "원하면 언제든 사람" 병행 체제로 수정
구현 선택	OpenAI 모델 기반 상담. 사내 정책 · 주문 정보를 근거로 답하는 RAG형 구조
품질과 위험	속도는 개선, 복잡한 문의의 품질 저하 · 고객 불만 보도 → 사람 상담원 재채용
수익 구조	2024년 이익 개선 약 4천만 달러 추정 vs 상담 품질 · 브랜드 비용

토론 질문 여러분이라면 어느 지점에 통제 장치를 두겠는가?

상용 서비스 아키텍처

DAY 1 :: STAGE 2/5

PoC 도구와 상용 서비스 구조는 무엇이 다른가?

--theory · 60분

구분	PoC 도구 (Streamlit · Gradio · Colab)	상용 서비스 구조 (Vue.js + 인증 + REST API + 모델 서빙)
통신 방식	WebSocket 연결 + 조작마다 스크립트 재실행	REST API (요청 → JSON 응답, 상태 비저장)
서버-화면 관계	한 덩어리 (화면과 로직이 같은 파이썬 코드)	분리 (프론트엔드 / 인증 / AI 백엔드 / 모델)
인증 · 권한	기본 수준, 직접 붙여야 함	인증서버가 JWT 발급, 서비스는 토큰만 검증
확장 · 연동	세션이 서버 메모리에 묶여 확장이 까다로움	서비스별 독립 확장, 모바일 · 사내 시스템이 같은 API 호출
주 용도	모델 데모 · 내부 검증 · 사내 도구	외부 고객 서비스 · 운영 · 배포

! **이번 과정의 원칙:** 화면은 Vue.js, 인증은 Java 서버, AI 기능은 FastAPI 서비스로 분리한다. Streamlit · Colab은 PoC 검증용으로만 쓴다.

상용 서비스 아키텍처

DAY 1 :: STAGE 2/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | ④주제·설계 | ⑤착수

전체 상용 서비스 아키텍처 - 한 장으로 보기

ARCHITECTURE



상용 서비스 아키텍처

DAY 1 :: STAGE 2/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | ④주제·설계 | ⑤착수

요청 흐름 - 한 번 로그인하고, 이후에는 토큰으로

REQUEST FLOW

1 로그인
Vue.js → POST /login
(id · password)

2 JWT 발급
Java 인증서버
회원 DB 확인 → 서명

3 AI 요청
Authorization:
Bearer <JWT>

4 토큰 검증
AI 서비스가 서명 ·
만료만 확인

5 응답
답변 · 출처 · 음성
(실패 시 401)

HTTP 예시

```
POST /rag-query HTTP/1.1
Host: localhost:8001
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Content-Type: application/json

{"question": "최신 AI 규제 동향은?"}
```

왜 인증을 분리하는가

- 서비스가 늘어도 로그인 로직은 한 곳
- AI 서비스는 회원 DB에 접근하지 않음
- 토큰 만료 · 권한으로 접근 통제

상용 서비스 아키텍처

DAY 1 :: STAGE 2/5

서비스 요소기술 - 무엇으로 만드는가

TECH STACK

계층	기술 스택	역할	조별 작업
프론트엔드	Vue.js · HTML/CSS	반응형 UX, 로그인 · 질문 · 녹음 · 재생	도메인화
인증	Java 웹서버 · JWT · 회원 DB	회원가입 · 로그인, 토큰 발급	골격 사용
음성 서비스	FastAPI · Whisper · FFMPEG	STT(.webm → 텍스트), 오디오 변환	확장 트랙
LLM · TTS	OpenAI GPT · OpenAI TTS (nova)	통번역 · 답변 생성, 음성 합성(mp3)	골격 사용
RAG 서비스	FastAPI · LangChain	검색 → 근거 기반 답변 + 출처	직접 구현
데이터 · 벡터DB	PDF Reader · 네이버 API · FAISS	외부 문서 수집 · 청킹 · 임베딩 저장	직접 구현
배포	Docker Compose	서비스별 컨테이너 일괄 기동	직접 구성

상용 서비스 아키텍처

DAY 1 :: STAGE 2/5

서비스 특징과 장단점 - 무엇을 얻고 무엇을 감수하는가

TRADE-OFF

항목	장점	단점 (감수할 비용)
MSA (서비스 분리)	음성 · RAG 서비스를 독립적으로 개발 · 확장 · 교체	서비스 간 통신 · 설정 · 배포 관리 증가
인증서버 분리 (JWT)	로그인 로직 한 곳, 서비스는 토큰만 검증	토큰 만료 · 키 관리 · 로그아웃 처리 필요
멀티 모델 전략	로컬 Whisper(보안 · 비용) + Closed API(품질 · 속도)	로컬 모델 설치 · 자원 관리 필요
근거 기반 응답 (RAG)	출처로 검증 가능, 환각 억제, 최신 정보 반영	데이터 정제 · 청킹 품질에 크게 의존
모듈화된 구조	MVP로 부분 도입 → 음성 · Agent로 확장	전사 적용 시 표준화 필요

! 아키텍처 선택은 언제나 트레이드오프다. 기획서에는 "무엇을 얻기 위해 무엇을 감수했는가"를 한 줄로 적는다.

데모·연결

DAY 1 :: STAGE 3/5

데모 진행 - 코드를 나눠 주지 않고, 계층을 따라가며 본다

--demo · 120분

① 15분
프론트엔드
Vue.js 반응형 UX
상태 · 화면 갱신

② 25분
인증서버
Java 웹서버
로그인 · JWT

③ 40분
음성 통번역
Physical AI
STT → GPT → TTS

④ 40분
RAG 연결
External RAG를
인증 API로 연결

구분	교수 데모 (관찰)	학생 활동
①~③	완성된 서비스를 계층별로 시연, 요청 · 응답을 개발자 도구로 확인	관찰 노트: 요청 URL · 헤더 · 응답 JSON · 실패 시 동작
④	RAG 서비스를 인증 API에 연결하는 과정을 시연	골격에 /rag-query 연결 → 토큰 포함 호출 성공 확인

데모·연결

DAY 1 :: STAGE 3/5

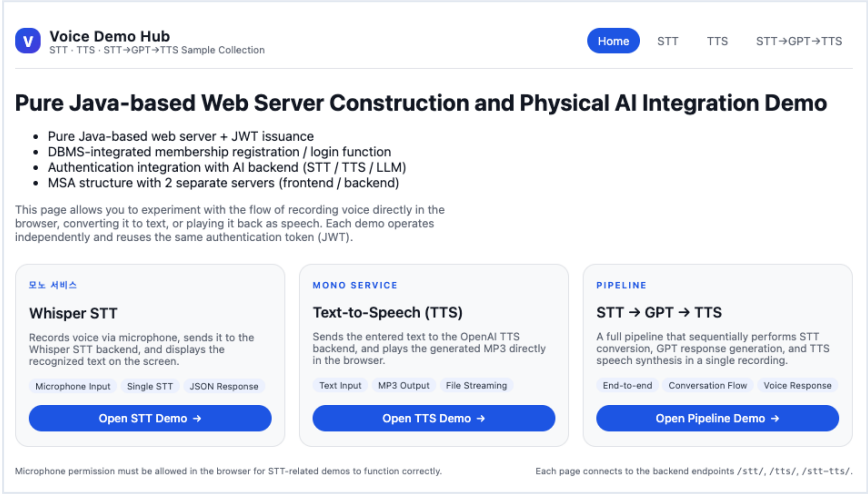
① 프론트엔드 - Vue.js 반응형 UX: 상태가 바뀌면 화면이 바뀐다

DEMO ①

DEV 상태 4가지와 반응성 (Vue 3)

```
<script setup>
import { ref, computed } from "vue"
const status = ref("idle")  // idle|loading|done|error
const answer = ref(null)
const busy = computed(() => status.value === "loading")
</script>

<template>
  <button :disabled="busy" @click="ask">질문</button>
  <p v-if="busy">답변을 만드는 중...</p>
  <p v-else-if="status==='error'">다시 시도해 주세요</p>
  <article v-else-if="answer">{{ answer.text }}
    <small v-for="s in answer.sources">{{ s }}</small>
  </article>
</template>
```



데모 포인트 허브 화면의 카드 3개가 화면 폭에 따라 3열 → 1열로 재배치되고, 버튼은 로딩 중 비활성화된다.

데모·연결

DAY 1 :: STAGE 3/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | ④주제·설계 | ⑤착수

① 프론트엔드 - AI 서비스 화면이 갖춰야 할 다섯 가지

DEMO ①

01

로딩 상태

3~5초 응답 동안
진행 표시 · 버튼 잠금

02

오류 안내

실패 · 타임아웃을
사용자 언어로

03

출처 표시

sources를 답변 아래
문서명 · 위치로

04

사람에게 넘기기

handoff: true면
담당자 연결 안내

05

인증 만료

401이면 로그인으로
토큰 갱신 · 재시도

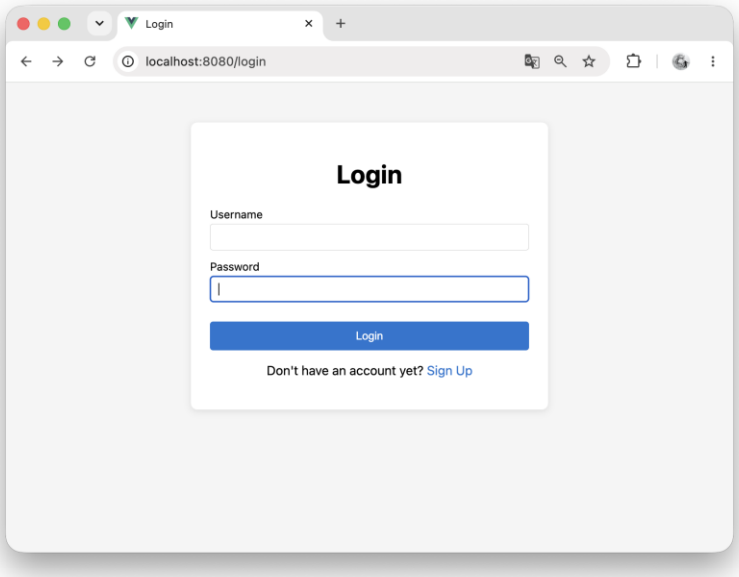
! **음성 기능 주의** 브라우저 마이크 권한이 필요하며 localhost 또는 https에서만 녹음된다.
모든 API 호출에 Authorization 헤더 첨부와 401 처리를 공통 함수로 넣는다.

데모·연결

DAY 1 :: STAGE 3/5

② 백엔드 - Java 인증서버: 회원 · 로그인 · JWT 발급

DEMO ②



localhost:8080/login · 심플 Java 인증서버 로그인

Pure Java 웹서버 + JWT

데모 허브: Java Web Server Construction & Physical AI Integration

- **웹서버:** 프레임워크 없이 순수 Java로 HTTP 처리
- **회원:** DBMS 연동 회원가입 · 로그인 (비밀번호 해시 저장)
- **토큰:** 로그인 성공 시 JWT 발급 → 이후 요청은 토큰으로 라우팅
- **연동:** AI 백엔드(STT · TTS · LLM)가 같은 토큰을 검증
- **구조:** 프론트엔드 / 백엔드 2개 서버로 분리된 MSA
- **상용에서는:** Spring Boot + Spring Security로 같은 흐름을 구성

데모·연결

DAY 1 :: STAGE 3/5

② 백엔드 - 발급은 Java, 검증은 각 AI 서비스

DEMO ②

Java 로그인 성공 → JWT 발급 (요약)

```
// 회원 확인 후
String token = Jwts.builder()
    .subject(user.getId())
    .claim("role", user.getRole())
    .issuedAt(new Date())
    .expiration(new Date(now + 3600_000))
    .signWith(key)          // .env의 비밀키
    .compact();

// 응답: {"accessToken": token}
```

Python FastAPI 서비스 - 토큰 검증 (요약)

```
def verify_jwt(auth: str = Header(...)):
    try:
        token = auth.removeprefix("Bearer ")
        return jwt.decode(token, SECRET,
                          algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(401, "로그인 필요")

@app.post("/rag-query")
def rag_query(req: AskRequest,
              user=Depends(verify_jwt)): ...
```

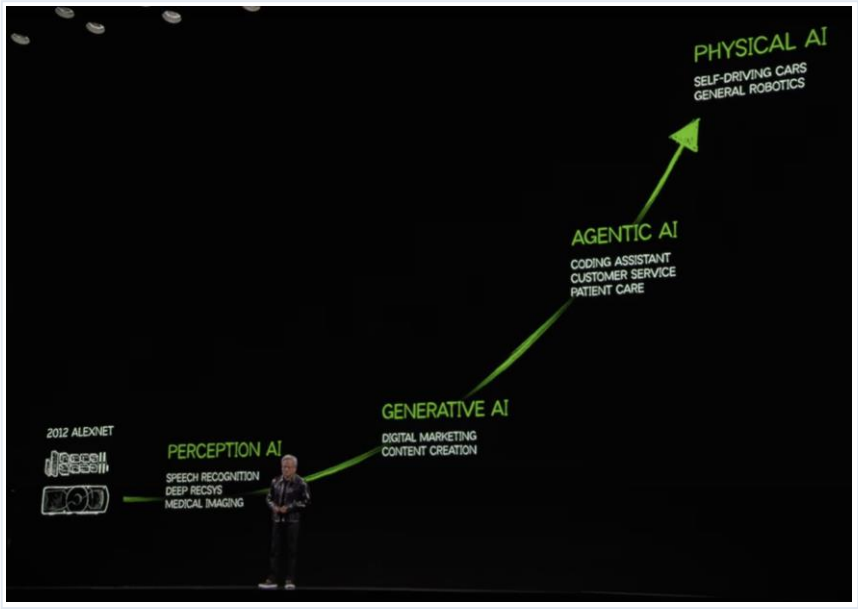
! 비밀키는 .env로만 주입하고, 토큰에는 반드시 만료 시간(exp)을 넣는다.

데모·연결

DAY 1 :: STAGE 3/5

③ Physical AI - 음성이 AI와 현장을 잇는 첫 번째 인터페이스

TREND



Perception → Generative → Agentic → Physical AI · Source: 젠슨 황 CES 2025 키노트

AI 음성 비서 에이전트

멀티모달 · 실시간 음성 기반 에이전트

- AI 연구 트렌드는 멀티모달이며, 사람의 소통 방식(말 · 듣기)을 모방하는 방향
- 음성 기반 실시간 AI 에이전트는 구성에 따라 확장 가능
- **확장 1:** 통 · 번역 (오늘 데모)
- **확장 2:** 회의록 작성 · 요약
- **확장 3:** RAG · 벡터DB 기반 사용자 지원 (④ 연결)

데모·연결

DAY 1 :: STAGE 3/5

③ 음성 통번역 앱 - STT → GPT → TTS 모델 서빙 파이프라인

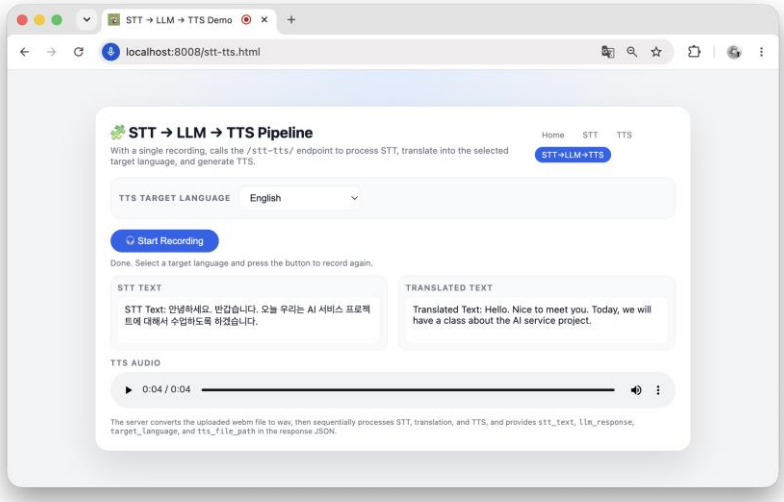
DEMO ③

STT
stt_service.py
마이크 .webm 업로드
.wav 변환 (FFMPEG)
Whisper → 텍스트
로컬 Open Weight

GPT 처리
processor_service.py
타겟 언어로 통번역
답변 생성
Closed API

TTS
tts_service.py
OpenAI TTS (nova)
output.mp3 저장
브라우저 재생
Closed API

- **실행 준비:** Python 3.11 · pip install -r requirements.txt
- **FFMPEG:** brew install ffmpeg / choco install ffmpeg
- **.env:** OPENAI_API_KEY (voice-ai-code 내부)



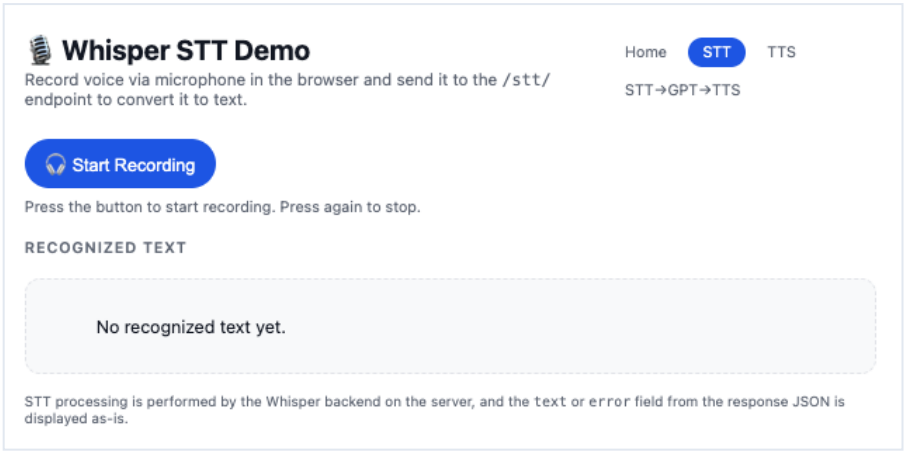
localhost:8008/stt-tts.html · STT → LLM → TTS 통합 파이프라인

데모·연결

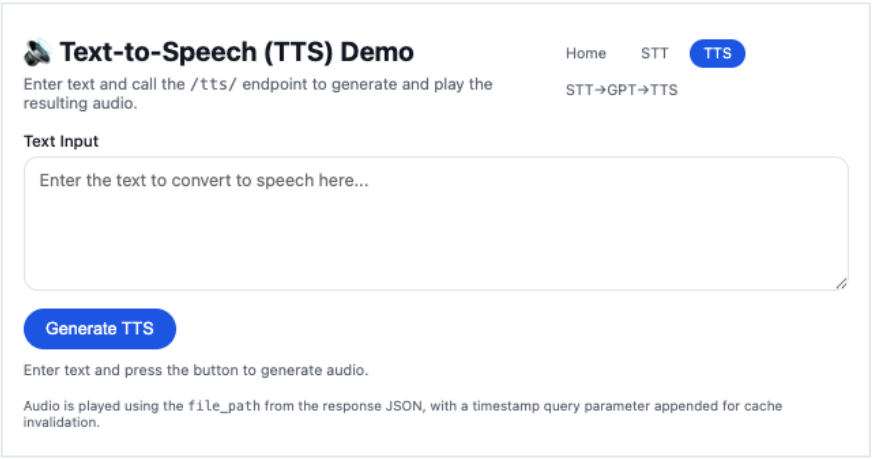
DAY 1 :: STAGE 3/5

③ 음성 통번역 앱 - 단계별로 쪼개서 확인한다

DEMO ③



localhost:8008/index.html · Whisper STT 데모 (POST /stt)



localhost:8008/tts-demo.html · TTS 데모 (POST /tts)

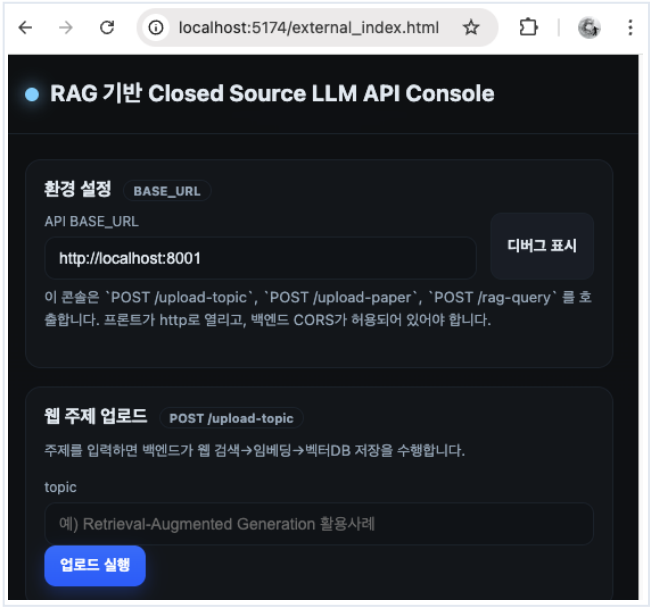
! **관찰 포인트** 요청마다 Authorization: Bearer 토큰이 붙고, 응답 JSON의 text · file_path가 화면에 그대로 반영된다.
다음 질문: 통번역 대신 인식된 텍스트를 RAG 서비스로 보내면? → ④ RAG 연결

데모·연결

DAY 1 :: STAGE 3/5

④ RAG 연결 - External RAG 서비스 (조별 직접 구현 대상)

DEMO ④



External · Closed LLM API Console

Endpoint	역할
POST /upload-paper	PDF 업로드 → 청킹 → 임베딩 → FAISS
POST /upload-topic	주제 → 웹 검색 → 임베딩 → FAISS
POST /rag-query	질문 → 검색 → 근거 답변 + 출처
GET /health	엔진 · 임베딩 준비 상태

POST /rag-query 응답 계약

```
{ "answer": "...", "sources": ["report.pdf p.12"],
  "needs_review": false, "handoff": false, "latency_ms": 2840 }
```

데모·연결

DAY 1 :: STAGE 3/5

④ RAG 연결 - 인증된 요청으로 잇고, 음성으로 묻는다

HANDS-ON ④

로그인
Java :8080
JWT 발급

→

음성 질문
Whisper STT
텍스트 변환

→

RAG 질의
/rag-query :8001
Bearer JWT

→

근거 답변
answer
+ sources

→

음성 답변
OpenAI TTS
mp3 재생

연결 실습 (학생)

- 골격 기동: 인증서버 · 프론트 셸 · RAG 서비스
- 로그인 → 개발자 도구에서 JWT 확인
- 토큰 없이 /rag-query → 401 확인
- 토큰 포함 /rag-query → 200 · sources 확인

음성 질의 RAG (교수 데모 · 요약)

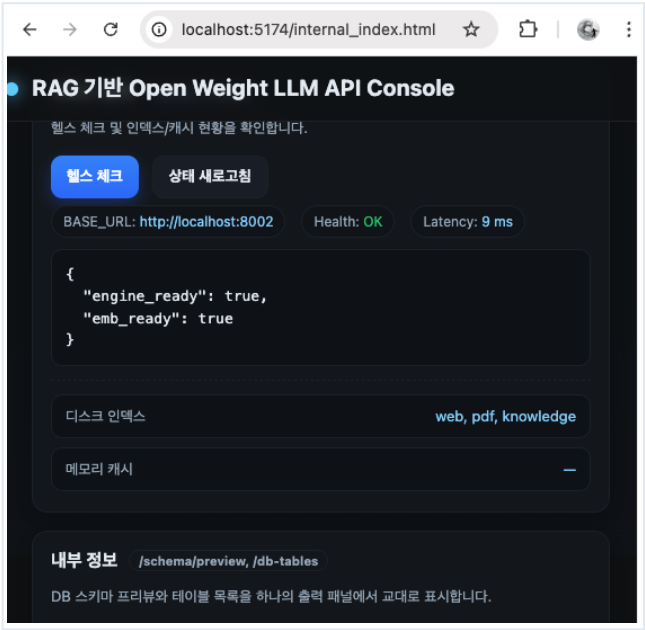
```
text = stt(audio)                    # Whisper
res = post(RAG_URL + "/rag-query",
           json={"question": text},
           headers={"Authorization": bearer})
mp3 = tts(res["answer"], voice="nova")
```

데모·연결

DAY 1 :: STAGE 3/5

참고 데모 - Internal RAG는 언제 쓰는가

DEMO · REFERENCE



Internal · Open Weight LLM Console

판단 기준	External (조별 필수)	Internal (데모 · 확장)
데이터	PDF 문서 · 웹 검색 결과	사내 DBMS (MariaDB)
보안 요구	공개 정보 위주	외부 API 전송 곤란
LLM	Closed LLM API (OpenAI)	Open Weight 직접 서빙 (Qwen)
운영 부담	토큰 비용 · 키 관리	GPU · 모델 로딩 · 후처리

! **한 줄 기준** "이 데이터가 회사 밖 API로 나가도 되는가?" 안 되면 Internal.
Internal은 확장 트랙(가산점)으로 도전 가능

데모·연결

DAY 1 :: STAGE 3/5

무엇을 받고, 무엇을 직접 만드는가

SCOPE

제공 (골격)

동작하는 뼈대

- Vue.js 프론트 셸 (로그인 · 질문 화면)
- Java 인증서버 (회원 · JWT)
- JWT 검증 함수 · API 계약 문서
- docker-compose.yml 골격

직접 구현 (필수)

조별 평가 대상

- 데이터 파이프라인 (PDF · Web → FAISS)
- RAG 서비스 /rag-query (인증 연동)
- 화면 도메인화 · 출처 표시
- 평가셋 · 개선 · 컨테이너 배포

확장 (선택 · 가산)

기본 E2E 완성 후

- 음성 연동 (STT → RAG → TTS)
- Internal RAG (Open Weight 서빙)
- Fine-tuning 또는 Tool Calling · Agent

주제 탐색·설계

DAY 1 :: STAGE 4/5

조별 주제 탐색과 설계 진행 방법

--workshop · 180분

STEP 1 · 30분
주제 탐색
대상 사용자 · 도메인 선정, 핵심 질문 10개

STEP 2 · 30분
데이터 선정
PDF · Web 출처 확정, 저작권 · 개인정보 점검

STEP 3 · 60분
서비스 설계
아키텍처 · 인증 흐름 · API 명세 · 역할 분담

STEP 4 · 60분
기획 리뷰
1장 기획서 발표 · 피드백 (체크포인트①)

- **주제 선정 기준:** 기대답변을 적을 수 있는 질문이 10개 이상 나오는 도메인, 다른 조와 겹치지 않게
- **데이터 기준:** 공개 PDF · 웹 정보 또는 직접 만든 가상 문서만 사용 - 실제 개인정보 · 저작권 문서 금지
- **범위 기준:** 골격(인증 · 프론트)은 그대로 쓰고, RAG 서비스와 화면 문구부터 우리 도메인으로
- 정답을 찾는 활동이 아니다 - 모르는 항목은 "확인이 필요한 질문"으로 적는다

주제 탐색·설계

DAY 1 :: STAGE 4/5

도메인 · 데이터 선택 - 질문의 답이 문서 안에 있는가?

SELECTION

판단 기준	좋은 선택	피해야 할 선택
데이터 출처	공개 PDF · 공공 안내 · 제품 매뉴얼 · 뉴스 검색	출처 불명 · 유료 구독 · 저작권 문서
답의 위치	핵심 질문 10개의 답이 문서 안에 있음	답이 문서 밖 상식 · 계산에 있음
문서 규모	PDF 5~30개 또는 웹 주제 3~5개	수백 개 문서 · 대용량 스캔 이미지
갱신 주기	변경이 드물거나 출처 날짜가 명확	실시간 변동 (주가 · 재고 등)
도메인 예시	병원 이용 안내 · 산업 동향 리서치 · 제품 매뉴얼 Q&A	개인 상담 · 의료 진단 · 법률 판단

! **한 줄 기준:** "질문 10개의 답을 우리가 모은 문서에서 찾아 줄 수 있는가?"

주제 탐색·설계

DAY 1 :: STAGE 4/5

조별 기획 활동지 - 서비스명 : [ ]

WORKSHEET

분석 틀	핵심 질문 (서비스에선? → 우리 구현에선?)	우리 조의 분석 · 설계
문제와 사용자	누구의 어떤 불편인가? → 핵심 질문 10개 · 기대답변 · 기존 방식은?	
AI의 역할	대체인가 보조인가? → needs_review · handoff 지점은?	
구현 선택	어떤 데이터로 차별화하는가? → PDF · Web 출처 · 임베딩 · LLM	
품질과 위험	어떤 오류가 위험한가? → 평가셋 · 출처 표시 · 마스킹은?	
수익 구조	누가 얼마를 내는가? → 요청당 원가와 줄이는 방법은?	
아키텍처 · 인증	계층 · 인증 흐름은? → 엔드포인트 · 요청/응답 스키마 · 401 처리	

주제 탐색·설계

DAY 1 :: STAGE 4/5

역할 분담표 - 계층별로 담당을 나눈다

ROLES

담당 파트	주요 작업	산출물 (커밋 · 결과물로 확인)	담당자
데이터 파이프라인	PDF · Web 수집 · 정제, 청킹 · 임베딩, FAISS 적재	적재 스크립트 · 인덱스 · 데이터 출처표	
RAG 서비스 · LLM	검색 → 근거 기반 답변, 프롬프트, Closed LLM 연동	RAG 체인 코드 · 프롬프트 · 평가 결과	
인증 연동 · API	JWT 검증, /rag-query 스키마 · 오류 처리 · 로그	API 명세 · Swagger · 미들웨어	
프론트엔드 · 배포	Vue.js 화면 도메인화, Docker Compose, README	화면 · compose 파일 · 배포 URL	

! 역할 분담표 = 기여 계수의 기준선. 체크포인트마다 지명된 팀원이 본인 파트를 직접 설명한다.

주제 탐색·설계

DAY 1 :: STAGE 4/5

①서비스 기획 | ②상용 아키텍처 | ③데모·연결 | **④주제·설계** | ⑤착수

체크포인트① 기획 리뷰 - 1장 기획서

CHECKPOINT ①

01

사용자 가치

대상 사용자의 불편과 중요성
핵심 질문 시나리오 10개 이상

02

데이터 · 솔루션

PDF · Web 출처와 선택 근거
저작권 · 개인정보 점검, 기능 범위

03

아키텍처 · API

계층 · 인증 흐름 다이어그램
/rag-query 요청 · 응답 명세

04

역할 분담표

팀원별 담당 파트와 산출물
(기여 계수 기준선)

! **평가 2-1 (10점)** 상(9~10): 4개 항목이 논리적으로 연결 · 중(5~8): 일반론 수준 · 하(0~4): 항목 누락
역할 분담표 미제출 시 최대 5점. 피드백은 다음 단계(착수) 전에 기획서에 반영한다.

프로젝트 착수

DAY 1 :: STAGE 5/5

체크포인트② 프로젝트 착수 - 로그인부터 RAG 호출까지 한 번 성공

CHECKPOINT ②

착수 체크리스트

- 조별 Git 저장소 생성, 골격 코드 반영
- .env 설정 · .gitignore 확인 (API 키 · JWT 비밀키)
- 인증서버(:8080) 로그인 성공, JWT 확인
- RAG 서비스(:8001) /health 200 확인
- 샘플 PDF 1개로 토큰 포함 /rag-query 성공
- Day2 작업 계획을 역할 분담표에 기록

E2E 기본 흐름 확인 (예시)

```
# 1) 로그인 → 토큰
TOKEN=$(curl -s -X POST localhost:8080/login \
  -d '{"username":"u1","password":"..."}' \
  | jq -r .accessToken)

# 2) 토큰 없이 → 401
curl -i -X POST localhost:8001/rag-query

# 3) 토큰 포함 → 200
curl -X POST localhost:8001/rag-query \
  -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"question":"..."}'
```

Day 2 · 계층별 구현과 End-to-End 연결

- ① 데이터 파이프라인 ② RAG 서비스
- ③ 인증 연동 REST API → 체크포인트③ 중간 점검
- ④ Vue.js 프론트엔드 연동 (확장: 음성 · Internal · Agent)
- ⑤ End-to-End 데모 → 체크포인트④

SK 주식회사 AX

Day 2 조별 구현

DAY 2 :: STAGE 1/5

①데이터 파이프라인 | ②RAG 서비스 | ③인증 연동 API | ④프론트 연동 | ⑤E2E 점검

Day 2 목표와 일정 - 계층을 하나씩 연결한다

SCHEDULE · 8H

순서	세션	주요 작업	시간	방식
1	데이터 파이프라인	데이터 수집 · 정제 → 청킹 · 임베딩 → VectorDB 적재	2	프로젝트
2	RAG 서비스	검색 → 근거 기반 답변, Closed LLM 연동, 프롬프트 규칙	1	프로젝트
3	인증 연동 API	JWT 검증 + /rag-query 스키마 · 출처 포함 응답 · 오류 처리	1	프로젝트
4	중간 점검	Swagger로 API 동작 시연, 근거 기반 답변 확인	1	체크포인트③
5	프론트엔드 연동	Vue.js 대화 화면 ↔ REST API 연동, 로딩 · 오류 처리	1	프로젝트
6	확장 트랙	음성 연동 · Internal RAG · Fine-tuning · Agent 중 택1	1	선택
7	Day2 점검	End-to-End 데모, Day3 평가 질문 초안 준비	1	체크포인트④

Day 2 조별 구현

DAY 2 :: STAGE 1/5

①데이터 파이프라인	②RAG 서비스	③인증 연동 API	④프론트 연동	⑤E2E 점검

데이터 파이프라인 - 수집 · 정제 · 청킹 · 적재

--build · 2H

PDF → VectorDB · /upload-paper

- 머리말 · 꼬리말 · 페이지 번호 등 잡음 정제 후 청킹
- 문서명 · 페이지를 메타데이터로 → faiss_db/pdf

Web → VectorDB · /upload-topic

- 주제 키워드 → 네이버 검색 API로 수집
- 본문 추출 · 중복 제거, URL · 수집일을 메타데이터로
- faiss_db/web 에 분리 저장

DEV 적재 스크립트 핵심 (LangChain)

```
splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(docs)

# 출처 메타데이터 = 응답의 sources
for c in chunks:
    c.metadata["source"] = c.metadata.get(
        "source", "unknown")

emb = HuggingFaceEmbeddings(
    model_name="BAAI/bge-m3")
db = FAISS.from_documents(chunks, emb)
db.save_local("faiss_db/pdf")
```

Day 2 조별 구현

DAY 2 :: STAGE 2/5

①데이터 파이프라인 | ②RAG 서비스 | ③인증 연동 API | ④프론트 연동 | ⑤E2E 점검

RAG 서비스 - 검색 결과를 근거로 답하게 만든다

--build · 1H

단계	하는 일	품질 규칙
Retriever	FAISS에서 질문과 가까운 청크 top-k 검색	0건이면 LLM 호출 안 함
Prompt	[문서] + 질문을 규칙과 함께 결합	문서에 없으면 "찾을 수 없습니다"
LLM	Closed LLM API로 답변 생성	타임아웃 · 재시도 · 키는 .env
후처리	답변 정리, 출처 목록 구성	sources · needs_review 채우기

! LLM 호출부를 OpenAI 호환 인터페이스로 두면, Internal(Open Weight 서빙) 전환은 .env 변경만으로 끝난다.

DEV RAG 서비스 함수 (요약)

```
client = OpenAI(
  base_url=os.getenv("LLM_BASE_URL"),
  api_key=os.getenv("LLM_API_KEY"))

def answer(q):
  docs = db.similarity_search(q, k=4)
  if not docs:
    return "찾을 수 없습니다", []
  ctx = join(docs)
  r = client.chat.completions.create(
    model=os.getenv("LLM_MODEL"),
    messages=prompt(ctx, q), timeout=30)
  srcs = [d.metadata["source"] for d in docs]
  return r.choices[0].message.content, srcs
```

Day 2 조별 구현

DAY 2 :: STAGE 3/5

①데이터 파이프라인 | ②RAG 서비스 | **③인증 연동 API** | ④프론트 연동 | ⑤E2E 점검

인증 연동 REST API - 토큰을 확인하고, 스키마로 답한다

--build · 1H

DEV FastAPI 엔드포인트 (요약)

```
class AskRequest(BaseModel):
    question: str = Field(min_length=2)
    source: Literal["pdf","web"]

class AskResponse(BaseModel):
    answer: str
    sources: list[str]
    needs_review: bool = False
    handoff: bool = False

@app.post("/rag-query", response_model=AskResponse)
def rag_query(req: AskRequest, user=Depends(verify_jwt)):
    docs = retriever(req.source).invoke(req.question)
    if not docs:   # 근거 없음 → LLM 호출 안 함
        return AskResponse(answer="찾을 수 없습니다",
                           sources=[], handoff=True)
    ...
```

API 구현 규칙

평가 2-2 ③ REST API 기준

- **인증:** Depends(verify_jwt) - 토큰 없거나 만료면 401
- **입력 검증:** Pydantic으로 길이 · 타입 · source 값 제한
- **근거 우선:** 검색 0건이면 LLM을 호출하지 않는다
- **출처 반환:** 문서명 · 위치를 sources에 담는다
- **오류 응답:** 타임아웃 · 모델 미준비는 503과 메시지
- **문서화:** /docs (Swagger)가 곧 API 명세 화면

Day 2 조별 구현

DAY 2 :: STAGE 3/5

①데이터 파이프라인 | ②RAG 서비스 | **③인증 연동 API** | ④프론트 연동 | ⑤E2E 점검

체크포인트③ 중간 점검 - 화면 없이 API만으로 시연한다

CHECKPOINT ③

점검 항목	시연 방법	통과 기준
인증 검증	토큰 없이 /rag-query 호출	401 반환, 토큰 포함 시 200
API 단위 동작	Swagger /docs에서 /rag-query 실행	200 응답, 스키마대로 JSON 반환
근거 기반 답변	핵심 질문 3개 실행	답변에 sources가 포함되고 내용이 일치
예외 질문	데이터에 없는 질문 1개 실행	"찾을 수 없음" 또는 handoff: true
담당자 설명	지명된 팀원이 본인 파트 설명	흐름과 설계 이유를 설명 (2-8 지표②)

! 계층이 제대로 분리되었다면, 화면이 없어도 서비스는 동작한다. 이것이 Streamlit 구조와의 결정적 차이다.

Day 2 조별 구현

DAY 2 :: STAGE 4/5

①데이터 파이프라인 | ②RAG 서비스 | ③인증 연동 API | **④프론트 연동** | ⑤E2E 점검

프론트엔드 연동 - Vue.js 화면이 REST API를 부른다

--build · 1H

DEV Vue.js 호출부 (요약)

```
const BASE_URL = import.meta.env.VITE_API_URL
const auth = {Authorization: `Bearer ${token.value}`}

async function ask() {
  loading.value = true; error.value = null
  try {
    const r = await fetch(`${BASE_URL}/rag-query`, {
      method: "POST",
      headers: {...auth, "Content-Type": "application/json"},
      body: JSON.stringify({question: q.value,
                            source: src.value})})
    if (r.status === 401) return router.push("/login")
    if (!r.ok) throw new Error(r.status)
    result.value = await r.json()  // answer, sources
  } catch (e) { error.value = "잠시 후 다시 시도" }
  finally { loading.value = false }
}
```

1. **로딩 표시**
   응답 3~5초 동안 진행 상태 표시
2. **오류 처리**
   실패 시 사용자 언어로 안내
3. **출처 표시**
   sources를 답변 아래 목록으로
4. **인증 만료**
   401이면 로그인 화면으로

Day 2 조별 구현

DAY 2 :: STAGE 4/5

①데이터 파이프라인 | ②RAG 서비스 | ③인증 연동 API | **④프론트 연동** | ⑤E2E 점검

확장 트랙 (선택) - 음성 · Internal · Fine-tuning · Agent

OPTIONAL · +5

음성 연동

Physical AI 음성 비서로 확장

- STT(Whisper) 결과를 /rag-query로
- 답변을 TTS(nova)로 재생
- JWT를 음성 요청에도 적용
- **권장:** Day1 데모 재사용

Internal RAG

사내 DB + Open Weight 서빙

- MariaDB → 임베딩 → FAISS
- Qwen 등 직접 서빙 (vLLM · Ollama)
- 응답 후처리 (태그 제거)
- **주의:** GPU · 로딩 시간

Fine-tuning · Agent

모델 · 도구 확장

- LoRA 어댑터 적용 전후 비교
- Tool Calling 도구 1개 이상
- 검색 ↔ 도구 호출 선택 흐름
- **주의:** 비용 · 지연 증가

! **평가 2-7 가산점 (+5)** 셋 중 1개 이상 구현 · 시연. 기본 E2E가 동작한 뒤에만 착수하고, 가산점에도 기여 계수가 적용된다.

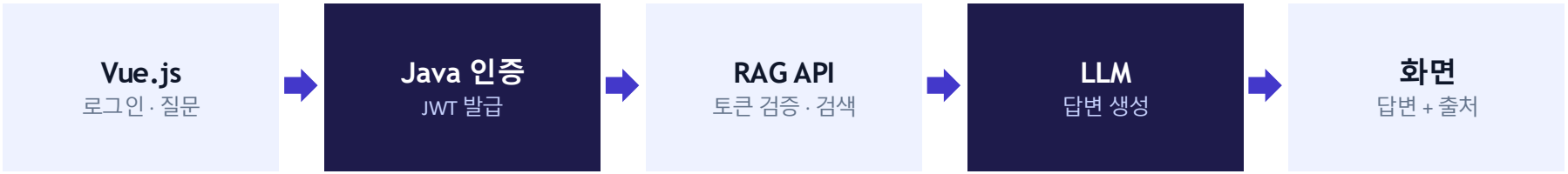
Day 2 조별 구현

DAY 2 :: STAGE 5/5

①데이터 파이프라인 | ②RAG 서비스 | ③인증 연동 API | ④프론트 연동 | ⑤E2E 점검

체크포인트④ End-to-End 데모와 Day3 준비

CHECKPOINT ④



E2E 데모 기준 (평가 2-2)

- 인증 → API → RAG → LLM 계층이 분리되어 동작
- 핵심 시나리오에 출처와 함께 정확히 답변
- 예외 질문에 "찾을 수 없음" · handoff
- 로딩 · 오류 처리가 화면에 보임

Day3 준비 과제

- 평가 질문 20개 + 기대답변 + 근거 문서 초안
- 예외 질문(문서에 없는 질문) 3개 이상 포함
- 오늘 발견한 실패 사례 메모
- docker-compose.yml 초안 확인

Day 3 · 품질 개선, 배포, 그리고 발표

- ① 평가셋 작성·측정 → 개선 루프
- ② Docker Compose 컨테이너 배포
- ③ 운영 관점 점검 (응답 시간 · 비용 · 로깅)
- ④ 최종 점검·퀴즈 → ⑤ 발표·회고

SK 주식회사 AX

Day 3 품질·배포·발표

DAY 3 :: STAGE 1/5

①평가·개선 | ②컨테이너 배포 | ③운영 점검 | ④최종 점검 | ⑤발표·회고

평가셋 작성 · 측정 - 품질을 숫자로 말한다

--eval · 1H

ID	질문	기대답변	근거 문서	정확	근거	환각
Q01	연차 이월 기준은?	최대 5일까지 이월	인사규정 12조	O	O	-
Q02	육아휴직 신청 기한은?	개시 30일 전	인사규정 21조	O	X	-
Q15	주차 지원금은 얼마인가?	(문서에 없음)	-	O	-	-
…	…	…	…			

정확성

기대답변과 같은 내용인가

근거성

출처 문서가 실제로 그 내용을 담는가

환각

문서에 없는 내용을 지어냈는가

Day 3 품질·배포·발표

DAY 3 :: STAGE 1/5

①평가·개선 | ②컨테이너 배포 | ③운영 점검 | ④최종 점검 | ⑤발표·회고

개선 루프 - 원인 단계를 찾아 하나씩 바꾼다

`--improve · 1H`

실패 현상	의심할 단계	개선 조치
관련 문서가 검색되지 않음	청킹 · 임베딩 · 검색	청크 크기 · 중첩 조정, top-k 증가, 메타데이터 필터
출처는 맞는데 답이 엉뚱함	프롬프트 · 생성	질문-근거 결합 방식 수정, 답변 규칙 명시
문서에 없는 내용을 지어냄 (환각)	프롬프트 · 검색 0건 처리	"찾을 수 없음" 규칙, 0건이면 LLM 호출 차단
질문 표현이 바뀌면 답이 달라짐	임베딩 · 질의 전처리	질의 재작성, 동의어 · 약어 사전
답은 맞는데 출처가 비어 있음	적재 · 메타데이터	청크마다 문서명 · 페이지 · URL 메타데이터 부여

! **규칙: 한 번에 하나만 바꾸고, 같은 평가셋으로 다시 잰다.**
발표용 기록 예: 정답률 12/20 → 17/20 (청크 500→300, top-k 3→5)

Day 3 품질·배포·발표

DAY 3 :: STAGE 2/5

컨테이너화 · 배포 - docker compose up 한 번으로

--deploy · 1H

docker-compose.yml (예시 골격)

```
services:
  db:                    # 회원 DB
    image: mariadb:11
    env_file: .env
  auth:                  # Java 인증서버 (골격)
    build: ./auth-server
    ports: ["8080:8080"]
    environment: { DB_HOST: db }   # localhost 아님
    depends_on: [db]
  rag:                   # 조별 RAG 서비스
    build: ./rag-service
    ports: ["8001:8001"]
    env_file: .env           # LLM 키 · JWT 비밀키
    volumes: ["./faiss_db:/app/faiss_db"]
  web:                   # Vue.js
    build: ./frontend
    ports: ["5174:80"]
```

배포 체크리스트 (평가 2-4)

- docker compose up -d 로 전체 기동
- 서비스 간 호출은 서비스 이름으로 (auth · rag · db)
- LLM 키 · JWT 비밀키는 .env로만 주입
- 배포 URL 또는 실행 영상 확보
- README: 실행 방법 · compose · 데이터 출처 · 평가 결과 · 팀원별 담당

Day 3 품질·배포·발표

DAY 3 :: STAGE 3/5

①평가·개선 | ②컨테이너 배포 | ③운영 점검 | ④최종 점검 | ⑤발표·회고

운영 관점 점검 - 데모가 아니라 매일 돌아가는 서비스로

--ops · 1H

01

응답 시간

평균 · 최대 latency
목표 3~5초 이내

02

토큰 비용

요청당 원가
월 예상 비용 계산

03

로깅

경로 · 지연 · 토큰
원문 · 개인정보 마스킹

04

예외 처리

타임아웃 · 모델 미준비
503 · 재시도 · 안내

DEV 지연 · 토큰 로그 미들웨어

```
@app.middleware("http")
async def ops_logger(request, call_next):
    t0 = time.perf_counter()
    resp = await call_next(request)
    log.info({
      "path": request.url.path,
      "status": resp.status_code,
      "latency_ms": round(
         (time.perf_counter()-t0)*1000),
      "in_tok": getattr(request.state,"in_tok",0),
      "out_tok": getattr(request.state,"out_tok",0)
    })  # 질문 원문은 저장하지 않음
    return resp
```

Day 3 품질·배포·발표

DAY 3 :: STAGE 4/5

①평가·개선 | ②컨테이너 배포 | ③운영 점검 | **④최종 점검** | ⑤발표·회고

체크포인트⑤ 최종 점검 - 제출물 규격과 개인 퀴즈

CHECKPOINT ⑤

제출물	규격	확인
서비스 데모	배포 URL 또는 실행 영상	
아키텍처 · API	계층 · 인증 흐름 다이어그램, API 명세 (Swagger 캡처 가능)	
발표 자료	10분 내외 · 문제 → 해결 → 결과(평가 수치) → 한계	
코드 저장소	README: 실행 방법 · Docker Compose · 데이터 출처 · 평가셋과 개선 결과	
평가셋	질문–기대답변 20쌍 이상, 판정 결과, 개선 전후 비교	

QUIZ

개인 퀴즈 · 20분

15문항 × 2점 = 30점

① 상용 아키텍처 · 인증 (JWT · MSA)
② RAG · 모델 서빙과 품질 진단
③ 평가 · 개선 및 배포 · 운영

용어 암기가 아닌 상황 진단형

Day 3 품질·배포·발표

DAY 3 :: STAGE 5/5

① 평가·개선 | ② 컨테이너 배포 | ③ 운영 점검 | ④ 최종 점검 | **⑤ 발표·회고**

최종 발표 · 라이브 데모 - 10분 안에 증명한다

`--present · 2H`

1분	2분	3분	2분	2분
문제	**아키텍처**	**라이브 데모**	**결과**	**한계 · 다음**
누구의 어떤 불편	계층 · 데이터 흐름 · 배포	핵심 + 예외 질문	평가 수치 · 전후 비교	상용화 개선 방향

- **아키텍처 구성도 필수:** Vue.js → Java 인증(JWT) → RAG API(VectorDB 검색) → LLM → 응답(출처), 데이터 파이프라인과 배포 구성 포함
- **개별 질의응답:** 팀원별 1문항, 본인 파트의 구현 내용과 이유를 설명 (기여 계수 지표③)
- **데모 실패 대비:** 실행 영상을 미리 준비하되, 라이브 시연을 먼저 시도한다

Day 3 품질·배포·발표

DAY 3 :: STAGE 5/5

①평가·개선 | ②컨테이너 배포 | ③운영 점검 | ④최종 점검 | **⑤발표·회고**

회고와 정리 - PoC에서 상용 서비스로

`--wrap-up · 1H`

팀 회고 (KPT)

10분, 팀원 전원 한 줄씩

- Keep: 잘 된 설계 결정 하나
- Problem: 가장 오래 막힌 지점
- Try: 상용화한다면 가장 먼저 바꿀 것

기억할 세 가지

3일의 핵심

- 모델은 출발점일 뿐, 차별점이 아니다
- 계층 · 인증을 분리해야 확장 · 교체 · 배포가 된다
- 품질 · 비용 · 책임은 코드(스키마 · 로그 · 평가셋)로 남긴다

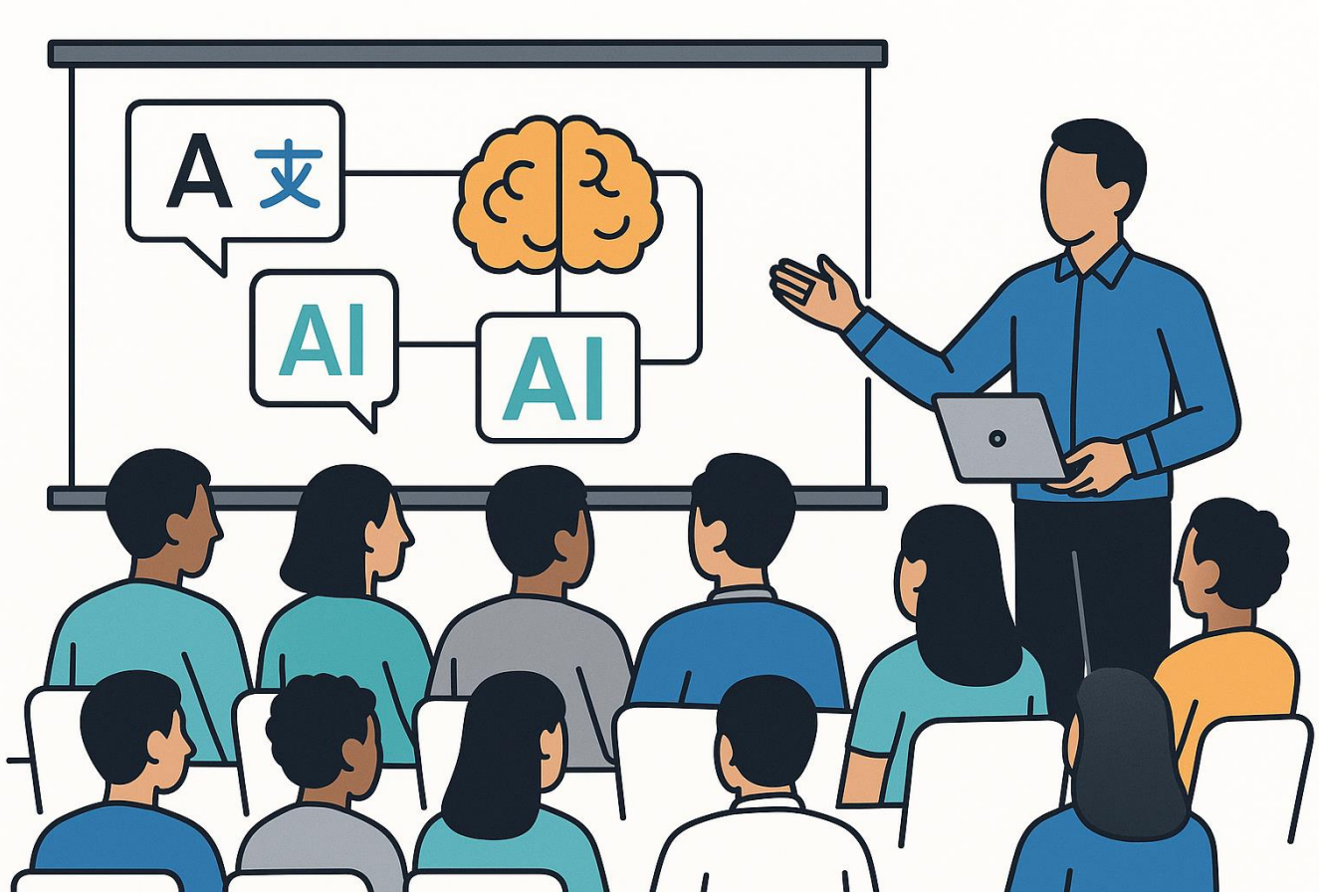
포트폴리오로 남기기

저장소가 곧 이력서

- README에 아키텍처 · 평가 수치
- 실행 영상 · 배포 URL
- 본인 담당 파트와 설계 이유

AI 서비스의 성패는 모델이 아니라 문제 정의와 운영에서 갈린다

마무리



현장 인터페이스에서 AI 가치 구현까지 · 수고하셨습니다